GOOGLE CLOUD

Host a Web App on Compute Engine

A hands-on deployment, scaling, load balancing, and update guide

Lab	Level	Duration	Credit
GSP662	Introductory	50 minutes	1

From source code to a resilient web platform

FINAL OUTCOME

A working Fancy Store application running on managed instance groups with load balancing, autohealing, autoscaling, Cloud CDN, and rolling updates.

Lab Overview

Compute Engine provides fine-grained control over virtual machines, networking, health checks, and load balancing. In this lab, you deploy the Fancy Store e-commerce sample, separate traffic into frontend and backend services, and evolve the deployment into a resilient, scalable architecture.

What you will learn

- Create Compute Engine virtual machine instances.
- Create instance templates from source instances.
- Build managed instance groups for frontend and backend services.
- Configure autohealing and load-balancer health checks.
- Create a global HTTP load balancer with path-based routing.
- Configure autoscaling and enable Cloud CDN caching.
- Roll out a new instance template and website update.
- Simulate an application failure and observe automatic repair.

Target architecture

Layer	Google Cloud resource	Purpose
Entry point	Global forwarding rule + target HTTP proxy	Accept public traffic on port 80.
Routing	URL map	Send / to frontend and /api paths to backend services.
Frontend	fancy-fe-mig	Serve the React frontend on port 8080.
Backend	fancy-be-mig	Serve Orders on 8081 and Products on 8082.
Resilience	Health checks + autohealing	Remove unhealthy backends and recreate failed VMs.
Scale	Autoscalers + Cloud CDN	Scale by utilization and cache frontend content.
Artifacts	Cloud Storage bucket	Store startup script and application source.

LAB SAFETY

Use only the temporary student credentials. Open the lab in an Incognito or private window, do not add recovery options or 2-step verification, and do not start a free trial. Using a personal account can incur charges.

Variable convention

Placeholder	Replace with
PROJECT_ID	The temporary lab project ID.
ZONE	The zone assigned by the lab.

Placeholder	Replace with
REGION	The region assigned by the lab.
BACKEND_ADDRESS	External IP of the backend VM.
LB_IP	Global external IP of the load balancer.
VM_NAME	Name of a managed frontend VM.

Setup and Cloud Shell

Start the lab and sign in

1. Click Start Lab and note the countdown timer. The lab cannot be paused.
2. Open the Google Cloud console in an Incognito/private window.
3. Choose Use Another Account if an account selector appears.
4. Sign in with the temporary username and password from the lab panel.
5. Accept the terms. Do not configure recovery options, two-factor authentication, or a free trial.
6. Use the Navigation menu or console search field to locate Google Cloud services.

Activate Cloud Shell

7. Click Activate Cloud Shell in the console toolbar.
8. Continue through the information dialog and authorize Cloud Shell.
9. Confirm that the active project is the lab project.

```
gcloud auth list
gcloud config list project
```

Set the default zone and region

```
gcloud config set compute/zone "ZONE"
export ZONE=$(gcloud config get compute/zone)

gcloud config set compute/region "REGION"
export REGION=$(gcloud config get compute/region)
```

COMMAND HYGIENE

Replace placeholders exactly. Keep startup scripts in Linux LF line-ending format, not CRLF.

Task 1 | Enable the Compute Engine API

Enable the API required to create and manage VM infrastructure.

```
gcloud services enable compute.googleapis.com
```

- ☐ The command completes without an error.
- ☐ Compute Engine resources can now be created.

Task 2 | Create a Cloud Storage Bucket

Create a bucket for the startup script and application code.

```
gcloud storage buckets create gs://fancy-store-PROJECT_ID
```

NAMING

Cloud Storage bucket names are globally unique. In the lab, append the assigned project ID exactly as instructed.

☐ Run Check my progress for “Create a Cloud Storage bucket”.

Task 3 | Clone and Test the Fancy Store

Download the sample application, build it, and preview it in Cloud Shell.

```
git clone https://github.com/googlecodelabs/monolith-to-microservices.git
cd ~/monolith-to-microservices
./setup.sh
```

After setup completes, ensure Cloud Shell is using the compatible Node.js version specified by the lab:

```
node_nvm_command
```

Start the microservices locally:

```
cd ~/monolith-to-microservices/microservices
npm start
```

Service	Port	Expected message
Frontend	8080	Frontend microservice listening on port 8080
Orders	8081	Orders microservice listening on port 8081
Products	8082	Products microservice listening on port 8082

- Use Web preview > Preview on port 8080 to view the frontend.
- Products and Orders are not yet exposed at this stage.
- Close the preview and press Ctrl+C to stop the server.

Task 4 | Create Compute Engine Instances

Prepare a reusable startup script, deploy backend and frontend VMs, and open the required ports.

4.1 Create the startup script

```
touch ~/monolith-to-microservices/startup-script.sh
```

Open the Cloud Shell Editor, locate the file, and add the following script. Replace PROJECT_ID before saving.

```
#!/bin/bash

# Install the logging agent.
curl -s "https://storage.googleapis.com/signals-agents/logging/google-fluentd-install.sh" | bash
service google-fluentd restart &

# Install dependencies.
apt-get update
apt-get install -yq ca-certificates git build-essential supervisor psmisc
apt-get install -y nodejs npm

# Copy the application from Cloud Storage.
mkdir /fancy-store
gcloud storage cp -r gs://fancy-store-PROJECT_ID/monolith-to-microservices/microservices/* /fancy-store/

# Install application dependencies.
cd /fancy-store/
npm install

# Create the service user.
useradd -m -d /home/nodeapp nodeapp
chown -R nodeapp:nodeapp /fancy-store

# Configure Supervisor.
cat >/etc/supervisor/conf.d/node-app.conf << EOF
[program:nodeapp]
directory=/fancy-store
command=npm start
autostart=true
autorestart=true
user=nodeapp
environment=HOME="/home/nodeapp",USER="nodeapp",NODE_ENV="production"
stdout_logfile=syslog
stderr_logfile=syslog
EOF

supervisorctl reread
supervisorctl update
```

WHAT THE SCRIPT DOES

Installs logging, Node.js, npm, and Supervisor; downloads the app from Cloud Storage; installs dependencies; runs the app as a dedicated user; and restarts it automatically when needed.

```
gcloud storage cp ~/monolith-to-microservices/startup-script.sh gs://fancy-store-PROJECT_ID
```

4.2 Copy the application to Cloud Storage

```
cd ~
rm -rf monolith-to-microservices/*/node_modules
gcloud storage cp -r monolith-to-microservices gs://fancy-store-PROJECT_ID/
```

4.3 Deploy the backend VM

```
gcloud compute instances create backend \
  --zone=ZONE \
  --machine-type=e2-standard-2 \
  --tags=backend \
  --image-family=DEBIAN_IMAGE_NAME \
  --image-project=debian-cloud \
  --metadata=startup-script-url=https://storage.googleapis.com/fancy-store-PROJECT_ID/startup-script.sh
```

4.4 Point the frontend to the backend

```
gcloud compute instances list
```

Copy the backend EXTERNAL_IP. In react-app/.env, replace localhost with that address:

```
REACT_APP_ORDERS_URL=http://BACKEND_ADDRESS:8081/api/orders
REACT_APP_PRODUCTS_URL=http://BACKEND_ADDRESS:8082/api/products
```

```
cd ~/monolith-to-microservices/react-app
npm install && npm run-script build
cd ~
rm -rf monolith-to-microservices/*/node_modules
gcloud storage cp -r monolith-to-microservices gs://fancy-store-PROJECT_ID/
```

4.5 Deploy the frontend VM and configure networking

```
gcloud compute instances create frontend \
  --zone=ZONE \
  --machine-type=e2-standard-2 \
  --tags=frontend \
  --image-family=DEBIAN_IMAGE_NAME \
  --image-project=debian-cloud \
  --metadata=startup-script-url=https://storage.googleapis.com/fancy-store-PROJECT_ID/startup-script.sh

gcloud compute firewall-rules create fw-fe \
  --allow tcp:8080 \
  --target-tags=frontend

gcloud compute firewall-rules create fw-be \
  --allow tcp:8081-8082 \
  --target-tags=backend
```

```
gcloud compute instances list
```

Open the frontend at http://FRONTEND_ADDRESS:8080 and verify the Home, Products, and Orders pages.

- ☐ The frontend page loads.
- ☐ Products and Orders work.
- ☐ Run Check my progress for deployment and networking.

Task 5 | Create Managed Instance Groups

Convert the source VMs into templates, create frontend and backend MIGs, and enable autohealing.

5.1 Create instance templates

```
gcloud compute instances stop frontend --zone=ZONE
gcloud compute instances stop backend --zone=ZONE

gcloud compute instance-templates create fancy-fe \
  --source-instance-zone=ZONE \
  --source-instance=frontend

gcloud compute instance-templates create fancy-be \
  --source-instance-zone=ZONE \
  --source-instance=backend

gcloud compute instance-templates list

gcloud compute instances delete backend --zone=ZONE
```

5.2 Create and configure MIGs

```
gcloud compute instance-groups managed create fancy-fe-mig \
  --zone=ZONE --base-instance-name=fancy-fe --size=2 --template=fancy-fe

gcloud compute instance-groups managed create fancy-be-mig \
  --zone=ZONE --base-instance-name=fancy-be --size=2 --template=fancy-be

gcloud compute instance-groups set-named-ports fancy-fe-mig \
  --zone=ZONE --named-ports=frontend:8080

gcloud compute instance-groups set-named-ports fancy-be-mig \
  --zone=ZONE --named-ports=orders:8081,products:8082
```

5.3 Configure autohealing

```
gcloud compute health-checks create http fancy-fe-hc \
  --port=8080 --check-interval=30s --healthy-threshold=1 \
  --timeout=10s --unhealthy-threshold=3

gcloud compute health-checks create http fancy-be-hc \
  --port=8081 --request-path=/api/orders --check-interval=30s \
  --healthy-threshold=1 --timeout=10s --unhealthy-threshold=3

gcloud compute firewall-rules create allow-health-check \
  --allow=tcp:8080-8081 \
  --source-ranges=130.211.0.0/22,35.191.0.0/16 \
  --network=default

gcloud compute instance-groups managed update fancy-fe-mig \
  --zone=ZONE --health-check=fancy-fe-hc --initial-delay=300

gcloud compute instance-groups managed update fancy-be-mig \
  --zone=ZONE --health-check=fancy-be-hc --initial-delay=300
```

HEALTH CHECK DESIGN

Load-balancer checks should detect traffic-serving failures quickly. Autohealing checks should be more conservative because failure can trigger VM replacement.

Task 6 | Create the HTTP Load Balancer

Expose one public IP and route frontend, Orders, and Products requests by path.

6.1 Create load-balancer health checks

```
gcloud compute health-checks create http fancy-fe-frontend-hc \
  --request-path=/ --port=8080

gcloud compute health-checks create http fancy-be-orders-hc \
  --request-path=/api/orders --port=8081

gcloud compute health-checks create http fancy-be-products-hc \
  --request-path=/api/products --port=8082
```

6.2 Create backend services and attach MIGs

```
gcloud compute backend-services create fancy-fe-frontend \
  --health-checks=fancy-fe-frontend-hc --port-name=frontend --global

gcloud compute backend-services create fancy-be-orders \
  --health-checks=fancy-be-orders-hc --port-name=orders --global

gcloud compute backend-services create fancy-be-products \
  --health-checks=fancy-be-products-hc --port-name=products --global

gcloud compute backend-services add-backend fancy-fe-frontend \
  --instance-group-zone=ZONE --instance-group=fancy-fe-mig --global

gcloud compute backend-services add-backend fancy-be-orders \
  --instance-group-zone=ZONE --instance-group=fancy-be-mig --global

gcloud compute backend-services add-backend fancy-be-products \
  --instance-group-zone=ZONE --instance-group=fancy-be-mig --global
```

6.3 Configure URL routing and the public entry point

```
gcloud compute url-maps create fancy-map \
  --default-service=fancy-fe-frontend

gcloud compute url-maps add-path-matcher fancy-map \
  --default-service=fancy-fe-frontend \
  --path-matcher-name=orders \
  --path-rules="/api/orders=fancy-be-orders,/api/products=fancy-be-products"

gcloud compute target-http-proxies create fancy-proxy --url-map=fancy-map

gcloud compute forwarding-rules create fancy-http-rule \
  --global --target-http-proxy=fancy-proxy --ports=80
```

Request	Destination
/ and frontend assets	fancy-fe-frontend
/api/orders	fancy-be-orders

Request	Destination
/api/products	fancy-be-products

6.4 Update frontend configuration to use the load balancer

```
gcloud compute forwarding-rules list --global
```

```
REACT_APP_ORDERS_URL=http://LB_IP/api/orders
REACT_APP_PRODUCTS_URL=http://LB_IP/api/products
```

```
cd ~/monolith-to-microservices/react-app
npm install && npm run-script build
cd ~
rm -rf monolith-to-microservices/*/node_modules
gcloud storage cp -r monolith-to-microservices gs://fancy-store-PROJECT_ID/

gcloud compute instance-groups managed rolling-action replace fancy-fe-mig \
  --zone=ZONE --max-unavailable=100%
```

6.5 Verify frontend health

```
watch -n 2 gcloud compute backend-services get-health fancy-fe-frontend --global
```

- Wait until both frontend backends report HEALTHY.
- Press Ctrl+C to exit watch.
- Open http://LB_IP after retrieving the IP from the forwarding-rule list.

PRODUCTION NOTE

This lab uses HTTP to avoid certificate setup. Use HTTPS with an appropriate certificate for production traffic.

Task 7 | Scale Compute Engine

Add autoscaling policies and enable Cloud CDN for the frontend.

```
gcloud compute instance-groups managed set-autoscaling fancy-fe-mig \
  --zone=ZONE --max-num-replicas=2 \
  --target-load-balancing-utilization=0.60

gcloud compute instance-groups managed set-autoscaling fancy-be-mig \
  --zone=ZONE --max-num-replicas=2 \
  --target-load-balancing-utilization=0.60

gcloud compute backend-services update fancy-fe-frontend \
  --enable-cdn --global
```

The autoscalers target 60% load-balancing utilization. Cloud CDN allows Google Front Ends to return cacheable frontend content from cache when available.

- ☐ Autoscaling is configured on both MIGs.
- ☐ Cloud CDN is enabled for the frontend backend service.
- ☐ Run Check my progress for Scale Compute Engine.

Task 8 | Update the Website and Test Autohealing

Roll out a new machine type, publish a homepage change, and simulate failure recovery.

8.1 Create and roll out a new frontend template

```
gcloud compute instances set-machine-type frontend \
  --zone=ZONE --machine-type=e2-small

gcloud compute instance-templates create fancy-fe-new \
  --region=$REGION \
  --source-instance=frontend \
  --source-instance-zone=ZONE

gcloud compute instance-groups managed rolling-action start-update fancy-fe-mig \
  --zone=ZONE --version=template=fancy-fe-new

watch -n 2 gcloud compute instance-groups managed list-instances fancy-fe-mig \
  --zone=ZONE
```

Proceed when at least one VM is RUNNING, its ACTION is None, and its INSTANCE_TEMPLATE is fancy-fe-new. Then inspect its machine type:

```
gcloud compute instances describe VM_NAME --zone=ZONE | grep machineType
```

8.2 Publish the homepage change

```
cd ~/monolith-to-microservices/react-app/src/pages/Home
mv index.js.new index.js
cat ~/monolith-to-microservices/react-app/src/pages/Home/index.js

cd ~/monolith-to-microservices/react-app
npm install && npm run-script build

cd ~
rm -rf monolith-to-microservices/*/node_modules
gcloud storage cp -r monolith-to-microservices gs://fancy-store-PROJECT_ID/

gcloud compute instance-groups managed rolling-action replace fancy-fe-mig \
  --zone=ZONE --max-unavailable=100%
```

8.3 Validate the updated deployment

```
watch -n 2 gcloud compute backend-services get-health fancy-fe-frontend --global

gcloud compute forwarding-rules list --global
```

- Wait for both frontend instances to report HEALTHY.
- Press Ctrl+C.
- Open http://LB_IP and confirm the homepage update is visible.

8.4 Simulate an application failure

```
gcloud compute instance-groups list-instances fancy-fe-mig --zone=ZONE
```

```
gcloud compute ssh INSTANCE_NAME --zone=ZONE

sudo supervisorctl stop nodeapp; sudo killall node
exit

watch -n 2 gcloud compute operations list \
 --filter='operationType~compute.instances.repair.*'
```

A successful repair shows a compute.instances.repair.recreateInstance operation with STATUS DONE. The same activity can be monitored in Compute Engine > VM instances.

LAB COMPLETE

You have deployed, load balanced, scaled, updated, and failure-tested a web application on Compute Engine.

Troubleshooting Playbook

Symptom	Checks and corrective action
Frontend does not load	Confirm the VM is RUNNING, startup script completed, and firewall rule fw-fe allows TCP 8080.
Orders or Products fail	Check .env addresses, backend external IP or LB_IP, fw-be, and backend health checks.
MIG instances stay unhealthy	Verify named ports, request paths, startup script bucket path, and allow-health-check source ranges.
Startup script fails	Inspect serial-port or syslog output. Confirm the bucket name, project ID, Node.js installation, and LF line endings.
Load balancer returns errors	Confirm all backend services have attached instance groups and report healthy backends.
Rolling update appears stuck	Use list-instances to inspect ACTION, STATUS, and instance template; inspect health before retrying.
Website shows the old version	Confirm the build succeeded, new files were copied to the bucket, and the frontend MIG was replaced.

Fast verification commands

```
gcloud compute instances list
gcloud compute instance-templates list
gcloud compute instance-groups managed list
gcloud compute backend-services list
gcloud compute forwarding-rules list --global
gcloud compute backend-services get-health fancy-fe-frontend --global
```

Completion Checklist

- ☐ Compute Engine API enabled.
- ☐ Cloud Storage bucket created.
- ☐ Fancy Store cloned and tested in Cloud Shell.
- ☐ Startup script and source code copied to Cloud Storage.
- ☐ Frontend and backend Compute Engine instances deployed.
- ☐ Firewall rules created for application ports.
- ☐ Frontend and backend managed instance groups created.
- ☐ Named ports and autohealing health checks configured.
- ☐ Global HTTP load balancer created with path-based routing.
- ☐ Frontend configuration updated to use the load-balancer IP.
- ☐ Autoscaling and Cloud CDN enabled.
- ☐ New instance template rolled out.
- ☐ Homepage update published through a rolling replacement.
- ☐ Failure simulated and automatic repair observed.

Key takeaways

- Instance templates make VM configuration repeatable.
- Managed instance groups provide a consistent fleet and support autohealing, autoscaling, and rolling updates.
- Health checks serve different purposes for traffic routing and instance repair.
- A URL map can route multiple application paths through one public load-balanced IP.
- Cloud Storage can act as a simple artifact source for VM startup automation.
- Cloud CDN can reduce origin traffic for cacheable frontend content.

NEXT STEP

For a production design, add HTTPS, least-privilege service accounts, secret management, observability, regional or multi-regional resilience, and a controlled rolling-update policy.